



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche, Informatiche e Matematiche

Corso di Laurea Magistrale in Informatica (LM-18)

Implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche

Relatore:

Prof. Nicola Capodieci

Correlatore:

Prof. Fabio Pellacini

Candidato:

Simone Vaiasinni

Matricola 201711

ANNO ACCADEMICO 2025/2026

Abstract

Floorplan3D: implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche

L'abstract è il riassunto della tesi. Deve dare in poche righe (in genere 150-300 parole, una mezza pagina), una visione completa e autosufficiente del lavoro: problema, approccio, risultati, senza bisogno di leggere il resto.

1. Contesto e problema (2-3 righe) Qual è il problema generale? Perché è rilevante? Nel tuo caso: la difficoltà di ottenere modelli 3D di ambienti architettonici a partire da planimetrie 2D CAD, utili per rendering, visualizzazione, ecc.

2. La sfida specifica (1-2 righe) Cosa rende il problema difficile? Nel tuo caso: l'assenza di uno standard unico tra i modelli CAD, che rende difficile generalizzare un algoritmo.

3. Cosa hai fatto (il cuore dell'abstract, 4-6 righe) Qui descrivi Floorplan3D: un sistema/pipeline che, a partire da un file DXF con muri, porte e finestre, produce una mesh 3D esportata in GLTF. Vale la pena accennare, senza entrare troppo nei dettagli implementativi, alle fasi principali della pipeline (parsing, ricostruzione geometrica dei varchi, estrazione del grafo planare e delle facce, triangolazione ed estrusione), perché è proprio questo il contributo tecnico del lavoro.

4. Risultati/validazione (1-2 righe) Cosa hai ottenuto? Hai testato su più planimetrie reali? Hai fatto un confronto qualitativo, dei rendering finali in Blender, delle metriche sui tempi di elaborazione, sulla qualità della mesh? Anche un risultato qualitativo ("il sistema produce mesh corrette e prive di artefatti su planimetrie reali di complessità variabile") va benissimo se non hai metriche quantitative.

Indice

Abstract	ii
1 Introduzione	1
1.1 I problemi dei modelli CAD: eterogeneità e mancanza di standard	1
1.2 Assunzioni progettuali	2
2 Progettazione e implementazione della pipeline	4
2.1 Preprocessing	4
2.2 Parsing	7
2.3 Collasso dei vertici	10
2.4 Ricostruzione delle fessure	13
2.4.1 Ricostruzione porte	13
2.4.2 Ricostruzione finestre	15
2.5 Estrazione e classificazione delle facce	17
2.5.1 Grafo planare embedded e struttura dati Half-Edge	17
2.5.2 Costruzione del PSLG con CGAL	21
2.5.3 Classificazione delle facce	22
2.6 Triangolazione ed estrusione	24
2.6.1 Pavimento e soffitto	25
2.6.2 Muri	25
2.6.3 Porte	25
2.6.4 Finestre	26
2.7 Esportazione	26
2.7.1 Gestione degli asset tramite placeholder	27
3 Generazione della scena e rendering in Blender	29
3.1 Script Python di generazione scena Blender	29
3.2 Setup scena in Blender: materiali, luci, camera, Cycles	29

4	Risultati e valutazione	30
5	Conclusioni	31
	Ringraziamenti	32
	Riferimenti	33

Elenco delle Figure

2.1	Modello CAD di caso di studio aperto in LibreCAD.	4
2.2	Layer dei muri prima della correzione: si notano il layer ridondante WALL_EXTERNAL, i contorni aperti e i segmenti isolati.	5
2.3	Layer dei muri dopo la correzione.	6
2.4	Rappresentazione originale delle finestre: si notano il blocco composto da centinaia di segmenti, la sovrapposizione tra due finestre e una finestra che invade lo spazio interno.	6
2.5	Rappresentazione delle finestre dopo la correzione.	7
2.6	Plot dei segmenti	10
2.7	Spatial Hash	11
2.8	Entrambi i grafici sono identici. Il secondo contiene la metà dei vertici	13
2.9	Singolo segmento	15
2.10	Doppio segmento	15
2.11	Le porte adesso sono rappresentate da due segmenti paralleli	16
2.12	Nel nostro caso, con 15 sample per segmento e tolleranza $\varepsilon = 2$ otteniamo 8 cluster distinti corrispondenti alle 8 finestre presenti nel modello.	18
2.13	Ricostruzione completa dei varchi: muri, porte e finestre.	19
2.14	Grafo planare e non planare	20
2.15	Visualizzazione Half-edge	21
2.16	Visualizzazione facce	23

Elenco dei Codici

1. Introduzione

L'obiettivo del presente lavoro di tesi è quello di sviluppare un sistema in grado di generare automaticamente una mesh 3D partendo da un modello CAD 2D di una planimetria di una singola stanza o di un appartamento.

Per ogni modello CAD 2D che verrà letto in input dal sistema il risultato sarà una mesh 3D con vertici, indici, normali e coordinate di texture che possono essere immediatamente utilizzate per il rendering.

Il dominio del problema comprende i piani architettonici CAD 2D in formato DXF (Drawing Exchange Format). Per poter iniziare a lavorare sulla progettazione della pipeline di elaborazione è stato prima necessario raccogliere un dataset composto da modelli CAD 2D rappresentativi del dominio di applicazione, più di dieci modelli CAD sono stati raccolti da diversi siti web come `draftsperson.net`, `pincad.com` e `archweb.com`. Questi modelli CAD erano inizialmente disponibili nel formato DWG e sono quindi stati convertiti in formato DXF e poi controllati con un software di editing come LibreCAD.

Mentre DWG e DXF sono standard de facto nel campo del CAD sono diversi tra loro per quanto riguarda l'approccio alla codifica: DWG è un formato binario proprietario, più efficiente nello spazio ma meno leggibile in quanto può essere letto da poche applicazioni, mentre DXF è un formato basato sul testo (in ASCII) sviluppato in modo tale da non presentare problemi di interoperabilità con i software di terzi. [3] Tra i modelli disponibili ne sono stati selezionati alcuni come studi di caso.

1.1 I problemi dei modelli CAD: eterogeneità e mancanza di standard

Il formato DXF non fornisce direttamente poligoni, bensì un insieme di entità geometriche disconnesse tra loro, quali POLYLINE, LINE, ARC ecc., organizzate in layer e blocchi. È qui che emerge la prima criticità: non esiste alcuno standard che imponga una convenzione univoca sui nomi dei layer o dei blocchi. Un architetto può decidere di nominare i layer in un certo modo, e un altro architetto

in un modo diverso. Questo vale anche per i blocchi, che molto spesso presentano nomi privi di significato.

Un secondo problema riguarda la rappresentazione geometrica degli elementi. Per quanto riguarda i muri, ad esempio, un architetto può rappresentarle con un semplice contorno privo di spessore, mentre un altro le disegna con uno spessore definito. Alcuni utilizzano polilinee che definiscono contorni già chiusi, altri invece semplici segmenti che spesso si intersecano impropriamente tra loro e risultano aperti, facendo apparire la geometria come "rotta".

Non solo i muri, ma anche le finestre presentano una notevole variabilità: alcuni architetti le descrivono come blocchi complessi composti da centinaia di segmenti, altri come semplici segmenti che collegano due pareti, altri ancora come rettangoli definiti da quattro lati, con ulteriori segmenti interni per rappresentare il vetro.

Le porte, invece, tendono a essere rappresentate in modo più uniforme tra i vari modelli, tipicamente tramite archi; in alcuni modelli, tuttavia, in particolare per le porte d'ingresso, possono essere rappresentate tramite polilinee o segmenti anziché tramite un arco.

Questa eterogeneità è la ragione principale per cui non è possibile progettare un algoritmo generale in grado di elaborare correttamente qualsiasi modello CAD. È necessaria, a monte dell'elaborazione automatica, una fase di preprocessing volta a rendere il modello quanto più consistente possibile (Capitolo 2).

1.2 Assunzioni progettuali

Per affrontare queste criticità vengono fatte delle assunzioni ragionevoli, che consentono di semplificare il problema:

1. i layer e i blocchi relativi a pareti, porte e finestre devono avere nomi riconoscibili (ad esempio A-WALL, A-DOOR, A-WINDOW, BLOCK-WINDOW, BLOCK-DOOR)
2. i muri devono essere rappresentati come facce chiuse (poligoni), dotate di proprio spessore; eventuali *crack* o discontinuità nei contorni devono essere riparate manualmente prima dell'esecuzione mediante software di editing CAD (come LibreCAD);

3. le porte e finestre sono considerati buchi, o discontinuità, nelle pareti e verranno trattati come tali durante la fase di estrusione, con altezze differenziate

2. Progettazione e implementazione della pipeline

2.1 Preprocessing

Come discusso nel Capitolo 1, la fase di preprocessing ha l'obiettivo di rendere il modello CAD quanto più consistente possibile prima dell'elaborazione. Si interviene correggendo i layer, blocchi, geometrie e ridondanze.

Come caso di studio prendiamo il modello DXF selezionato e lo apriamo con LibreCAD per analizzarlo.

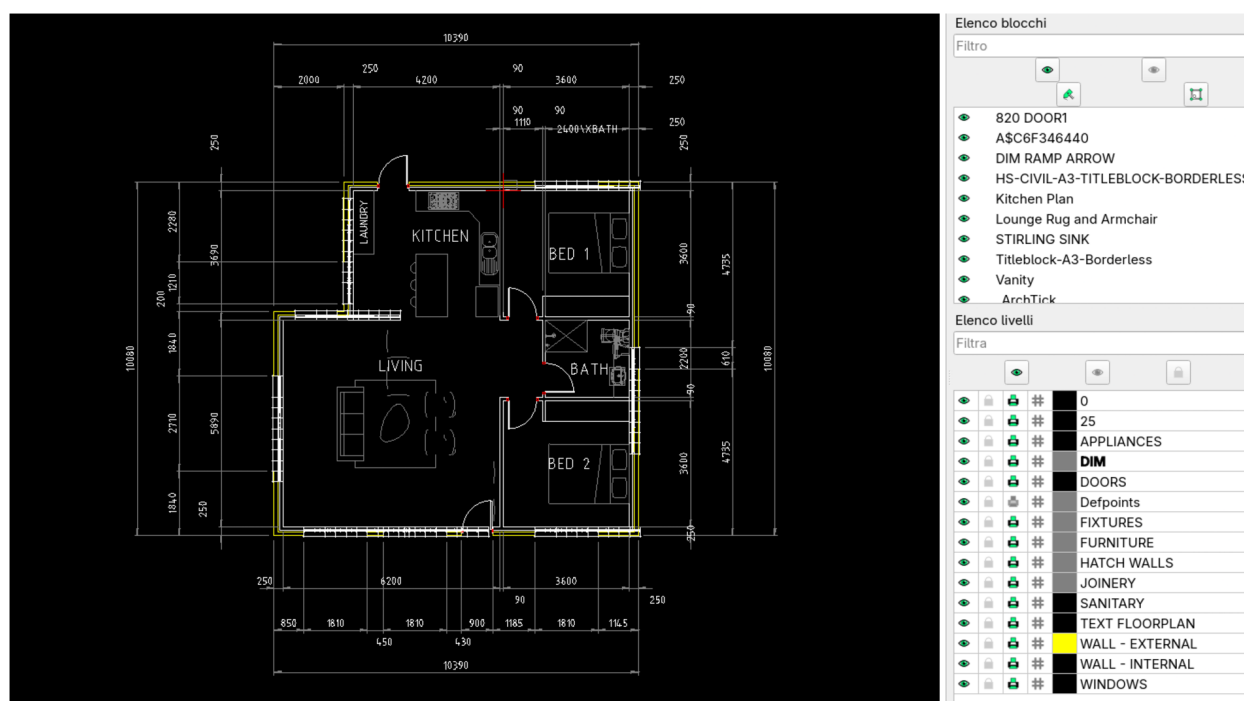


Figura 2.1: Modello CAD di caso di studio aperto in LibreCAD.

Isoliamo inizialmente solo i layer relativi ai muri e osserviamo che sono presenti due layer distinti: WALL EXTERNAL (evidenziato in giallo) e WALL INTERNAL. I segmenti del layer WALL EXTERNAL risultano ridondanti e possono essere rimossi senza alcun impatto sulla geometria finale. Osserviamo inoltre che le primitive dei muri non sono polilinee chiuse, bensì segmenti che formano

contorni aperti: la geometria risulta quindi incompleta e necessita di correzione. Sono infine presenti due ulteriori segmenti isolati, che costituiscono rumore e possono anch'essi essere rimossi.

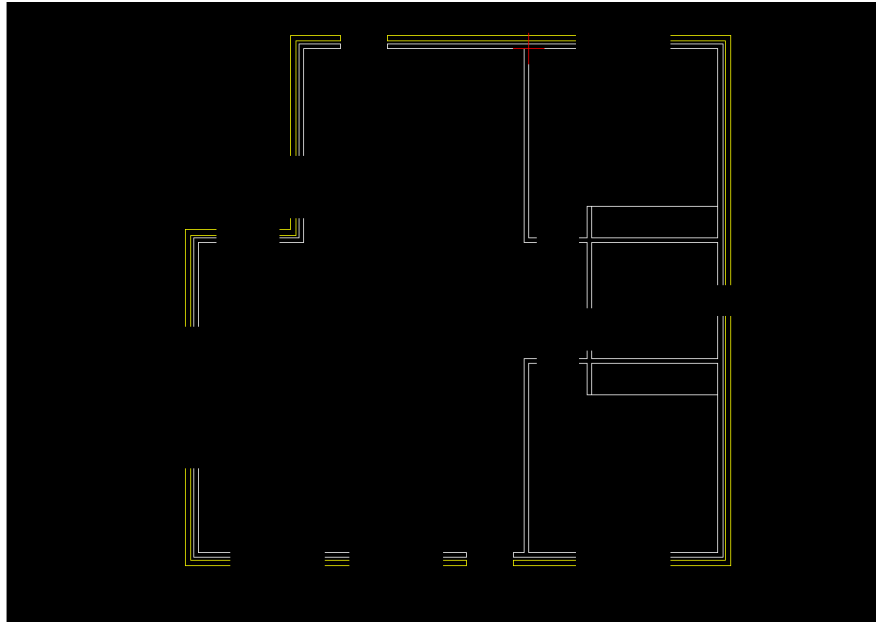


Figura 2.2: Layer dei muri prima della correzione: si notano il layer ridondante WALL_EXTERNAL, i contorni aperti e i segmenti isolati.

Dopo aver corretto questi problemi, otteniamo la rappresentazione dei muri mostrata in figura:

Nonostante i muri siano ancora separati tra loro, a causa della presenza di porte e finestre, quello che otteniamo è un insieme di poligoni chiusi. Analizziamo a questo punto le finestre: osserviamo come non siano rappresentate in una forma adeguata. Ogni finestra è descritta come un blocco (BLOCK) contenente centinaia di segmenti; inoltre, due finestre risultano sovrapposte tra loro, e due di queste fuoriescono addirittura dalle mura.

Dopodiché, i layer sono stati rinominati in modo coerente: A-WALL per i muri, A-DOOR per le porte e A-WINDOW per le finestre. Tutti gli altri layer, non essendo rilevanti per la ricostruzione della geometria, sono stati esclusi. Anche i blocchi sono stati rinominati: un blocco per la porta (BLOCK-DOOR-1) e quattro blocchi per le finestre (BLOCK-WINDOW-1, . . . , BLOCK-WINDOW-4).

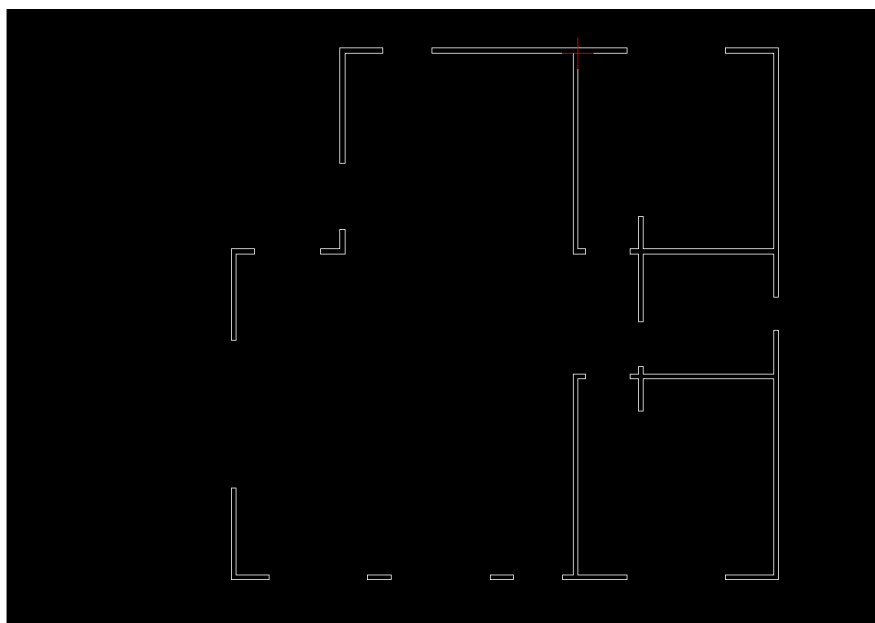


Figura 2.3: Layer dei muri dopo la correzione.

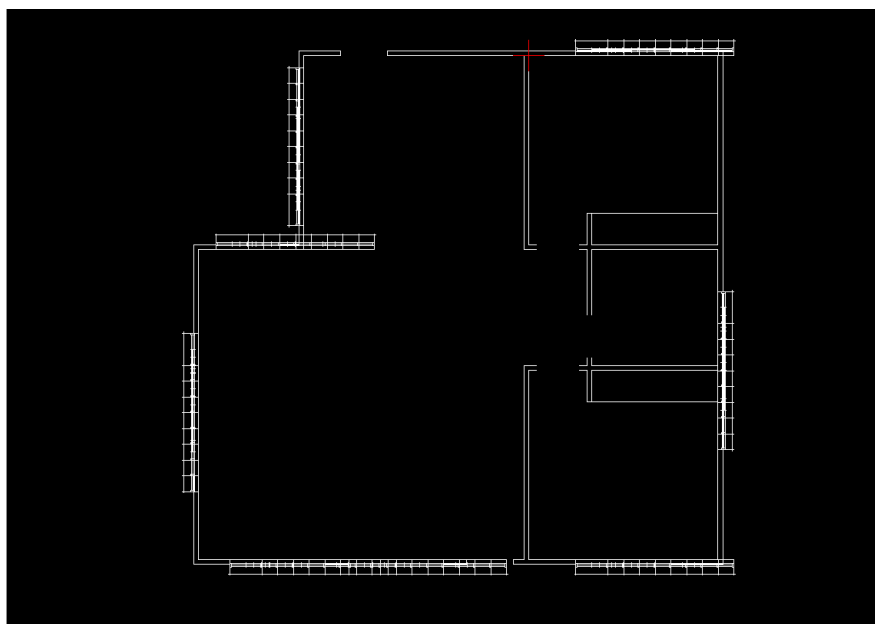


Figura 2.4: Rappresentazione originale delle finestre: si notano il blocco composto da centinaia di segmenti, la sovrapposizione tra due finestre e una finestra che invade lo spazio interno.

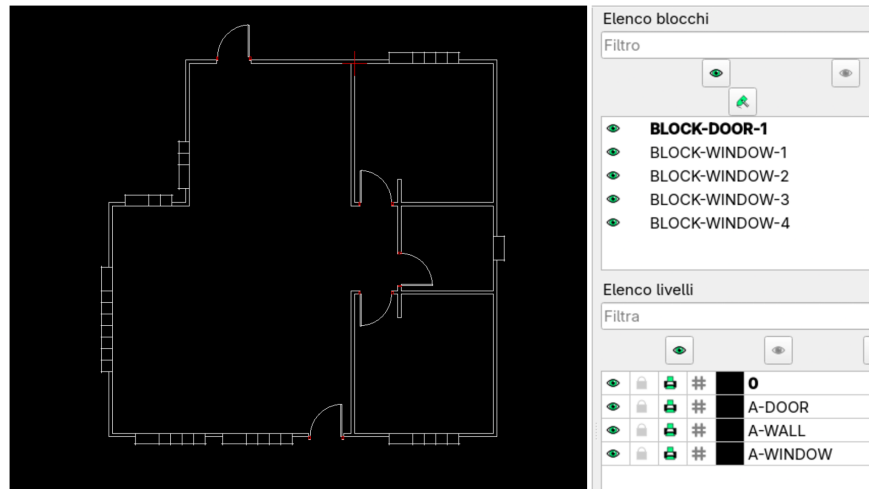


Figura 2.5: Rappresentazione delle finestre dopo la correzione.

2.2 Parsing

Il formato DXF organizza il disegno in primitive geometriche: LINE, POLYLINE, ARC, CIRCLE ecc. Queste entità sono raggruppate in layer che consentono di separare logicamente i diversi elementi del disegno (ad esempio muri, porte, finestre, arredi, testo). Oltre ai layer, il formato DXF supporta i blocchi (BLOCK), che non sono altro che componenti riutilizzabili. Sono comunemente utilizzati per rappresentare porte e finestre. Un blocco viene definito una volta all'interno del file e può essere istanziato più volte tramite entità INSERT, ciascuna con una propria posizione, scala e rotazione. Questa funzionalità è particolarmente utile per rappresentare elementi ripetuti, come porte, finestre o arredi, che compaiono più volte nella stessa planimetria.

L'obiettivo della fase di parsing è estrarre da queste primitive le informazioni geometriche necessarie alla costruzione della mesh, e collassare tutte le primitive sotto forma di segmenti per avere una rappresentazione uniforme.

Si consideri il modello CAD corretto durante il preprocessing, che presenta layer standardizzati (A-WALL, A-DOOR, A-WINDOW) e blocchi rinominati (BLOCK-DOOR-1, BLOCK-WINDOW-1, ..., BLOCK-WINDOW-4).

Il parsing viene realizzato in C++ mediante la libreria libdxfrw [2], che fornisce un'interfaccia per la lettura dei file DXF. La classe del parser implementa le callback necessarie per gestire i diversi tipi di entità:

```

enum class SegmentLayer : i32 { None=0, Wall, Window, Door };
struct Segment{ glm::dvec2 start, end; SegmentLayer layer; };

class DRWParser : public DRW_Interface
{
public:
    virtual void addLine(const DRW_Line& data) override;
    virtual void addLWPolyline(const DRW_LWPolyline& data) override;
    virtual void addArc(const DRW_Arc& data) override;
    virtual void addInsert(const DRW_Insert& data) override;
    virtual void addBlock(const DRW_Block& data) override;
    virtual void endBlock() override;

    virtual void addHatch([[maybe_unused]]const DRW_Hatch* data) override {}
    virtual void addCircle([[maybe_unused]]const DRW_Circle& data) override {}
    ...
    SegmentLayer classify_layer(std::string_view name);
    std::vector<Segment> walls, doors, windows;
};

```

Tra le callback messe a disposizione, quelle che ci interessano maggiormente sono:

- `addLine()`: viene invocata per ogni segmento (LINE) presente nel disegno. I due punti estremi vengono memorizzati direttamente come segmento;
- `addLWPolyline()`: per le polilinee (LWPolyline), si iterano i vertici della polilinea per generare una sequenza di segmenti consecutivi;
- `addArc()`: per gli archi (ARC), si estraggono il raggio, il punto base, l'angolo iniziale e l'angolo finale. A partire da questi dati, si calcolano i due punti estremi dell'arco, che corrispondono agli spigoli delle pareti; il segmento che li collega costituisce la rappresentazione della porta;

- `addInsert()`, `addBlock()`: questi metodi gestiscono le istanze dei blocchi. Le primitive all'interno di un blocco sono definite in coordinate locali; è necessario applicare una trasformazione per rappresentarli in world space.

Per distinguere i segmenti in base alla loro semantica, si utilizza il nome del layer:

```
SegmentLayer classify_layer(std::string_view name)
{
    if (name.compare("A-DOOR") == 0)    return SegmentLayer::Door;
    if (name.compare("A-WALL") == 0)    return SegmentLayer::Wall;
    if (name.compare("A-WINDOW") == 0)  return SegmentLayer::Window;
    return SegmentLayer::None;
}
```

Analogamente anche per i blocchi:

```
void DRWParser::addBlock(const DRW_Block& data)
{
    if(data.name.contains("BLOCK-DOOR") ||
        data.name.contains("BLOCK-WINDOW"))
    {}
}
```

Per questo motivo rinominare correttamente layer e blocchi è fondamentale.

Questa classificazione per layer e per blocco è fondamentale perché, come si vedrà successivamente, consente di trattare in modo differenziato pareti, porte e finestre durante la fase di estrusione.

Per verificare la correttezza del parsing, utilizziamo uno script Python per fare il plot del grafico:

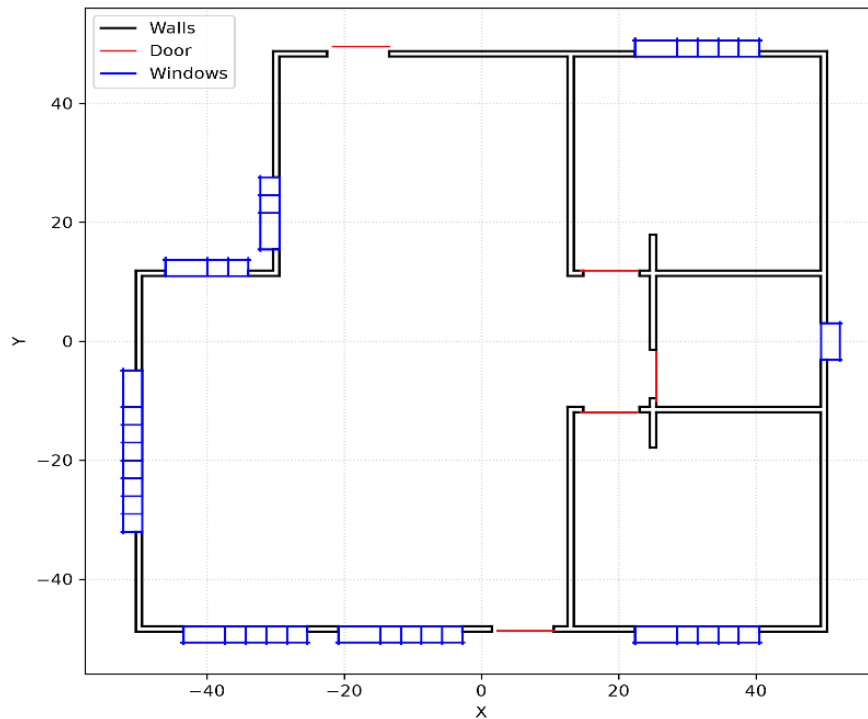


Figura 2.6: Plot dei segmenti

2.3 Collasso dei vertici

A questo punto il parser produce una raccolta di segmenti che non sono collegati tra loro: non abbiamo ancora una rappresentazione globale della geometria.

Nei modelli CAD è abbastanza comune che in uno stesso punto siano presenti più vertici sovrapposti, appartenenti a segmenti diversi. Il passo successivo consiste quindi nell'effettuare il collasso dei vertici, comunemente noto come *vertex snapping*. Questa operazione serve sia per ottimizzare la computazione, riducendo il numero di vertici da elaborare, sia perché la geometria dovrà in seguito essere rappresentata come grafo, il che richiede vertici univoci.

L'obiettivo è quindi prendere l'insieme dei segmenti e produrre una nuova rappresentazione in cui i vertici degli estremi che distano meno di ϵ l'uno dall'altro vengono collassati in un unico vertice rappresentativo.

Per risolvere questo problema è necessaria una struttura dati spaziale efficiente. Tra le opzioni possibili, come un K-D Tree, si è scelto di utilizzare un *hash spaziale*, concettualmente più semplice da implementare e ben adatto a questo caso d'uso. Un hash spaziale divide lo spazio 2D in

una griglia regolare, in cui ogni cella ha dimensione fissa pari a ϵ ; a ogni punto viene assegnata esattamente una cella in base alle sue coordinate (x,y). La proprietà chiave di questa struttura è che, se due punti distano meno di ϵ , essi si trovano necessariamente nella stessa cella oppure in una delle celle immediatamente adiacenti. Invece di confrontare ogni nuovo punto con tutti i punti già inseriti (complessità $O(N)$), è quindi sufficiente calcolare la cella di appartenenza del punto e controllare solo le 9 celle vicine (griglia 3×3), riducendo la complessità dell'operazione di snap a un tempo pressoché costante nella pratica.

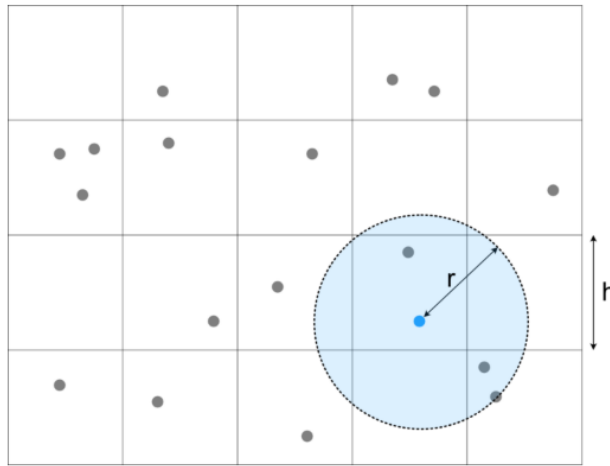


Figura 2.7: Spatial Hash

La dimensione della cella non è arbitraria: se le celle fossero più grandi (ad esempio 2ϵ), sarebbe comunque necessario controllare una griglia 3×3 , ma con un numero maggiore di punti per cella da confrontare. Se invece fossero più piccole (ad esempio $\epsilon/2$), sarebbe necessario estendere la ricerca a una griglia 5×5 (25 celle) per coprire correttamente il raggio ϵ . Per questo motivo, mantenere la dimensione della cella esattamente pari a ϵ e limitare la ricerca alla griglia 3×3 rappresenta il miglior compromesso tra memoria e velocità.

In questa fase è fondamentale tarare in modo adeguato il parametro ϵ : un valore troppo grande rischia di collasare erroneamente vertici distinti (ad esempio due angoli ravvicinati ma geometricamente reali), mentre un valore troppo piccolo lascia vertici duplicati che possono compromettere la costruzione del grafo nella fase successiva.

La classe che implementa questa struttura si presenta come segue:

```
class SpatialHash
{
public:
    SpatialHash(f64 epsilon = 1e-4) :
        m_epsilon{ epsilon}, m_epsilon_sq{ epsilon * epsilon } {}

    VertexId snap(glm::dvec2 p);
    VertexId find_nearest(glm::dvec2 p) const;

private:
    CellCoord get_cell(glm::dvec2 p);

    f64 m_epsilon, m_epsilon_sq;

    std::unordered_map<CellCoord, std::vector<VertexId>, CellCoordHash> m_grid;
    std::vector<glm::dvec2> m_vertices;
};
```

Il parametro ϵ definisce sia la dimensione della cella sia la tolleranza massima per il merging dei vertici. Il metodo principale è `snap()`, che:

- prende un punto p e cerca nella griglia 3×3 un vertice già esistente che disti meno di ϵ ;
- se lo trova, restituisce il `VertexId` del vertice esistente (il nuovo punto viene collassato su di esso e scartato);
- se non lo trova, crea un nuovo vertice, lo memorizza nella griglia e ne restituisce il nuovo `VertexId`.

Il metodo `find_nearest()`, invece, restituisce l'ID del vertice più vicino al punto p all'interno dell'intero hash. Questo metodo ci servirà più avanti quando dovremo ricostruire le fessure dei muri.

In questo esempio, con $\epsilon = 0.01$, il numero di vertici sui segmenti dei muri si riduce da 177 a 88, ovvero una riduzione di circa il 50%.

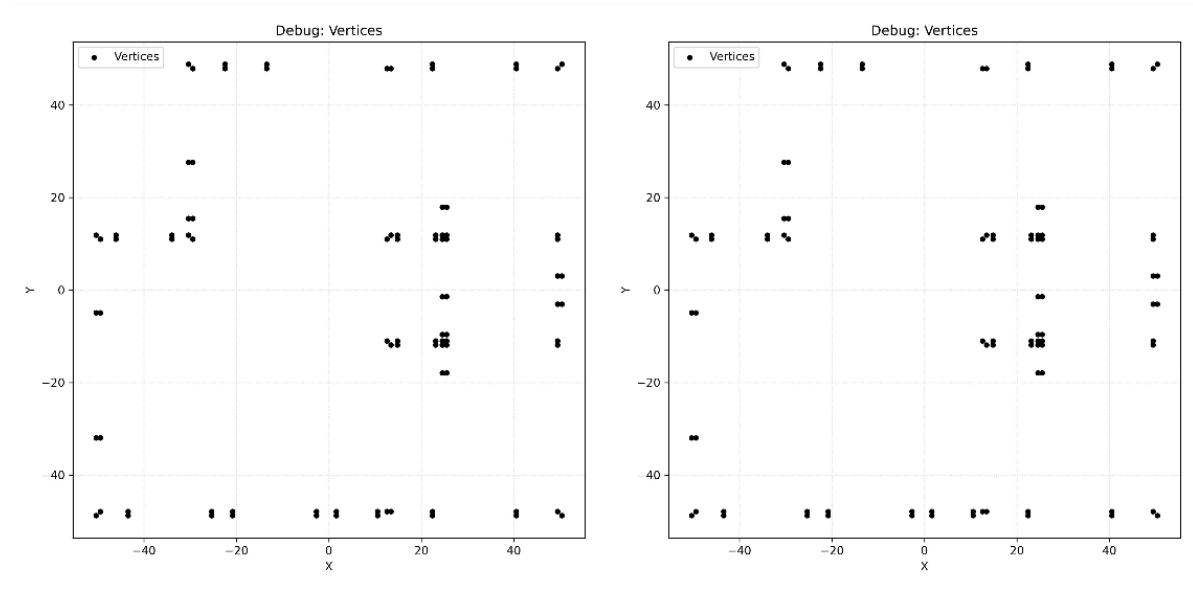


Figura 2.8: Entrambi i grafici sono identici. Il secondo contiene la metà dei vertici

2.4 Ricostruzione delle fessure

2.4.1 Ricostruzione porte

Consideriamo il caso più semplice, ovvero le porte. L'obiettivo è prendere i soli segmenti relativi alle porte e ricostruire, per ciascuno di essi, la faccia chiusa corrispondente, sfruttando la struttura spaziale e la lista degli archi (edge) costruite nella fase precedente.

Ricordiamo che la procedura di snapping è stata eseguita esclusivamente sui segmenti dei muri: al suo termine abbiamo quindi a disposizione l'hash spaziale e una lista di archi, dove ogni arco

è rappresentato da due indici di vertice e da un tag che ne descrive il tipo (WALL/DOOR/WINDOW). A questo punto, sia i vertici presenti nell'hash sia gli archi della lista appartengono esclusivamente ai muri:

```
struct Edge
{
    VertexId v1, v2;
    SegmentLayer layer;
};

struct SnappingResult
{
    SpatialHash hash;
    std::vector<Edge> edges;
};
```

L'idea è di iterare su ogni segmento delle porte e, per ciascuno, considerarne i due vertici estremi. Invece di creare due nuovi vertici, sfruttiamo il fatto che una porta collega sempre gli spigoli di due muri: tramite il metodo

```
VertexId find_nearest(glm::dvec2 p)
```

recuperiamo i due vertici dei muri corrispondenti a quella specifica porta, senza duplicarli, e aggiungiamo un nuovo arco tra di essi con tag DOOR.

Una porta, tuttavia, non è rappresentabile con un singolo segmento: è una faccia rettangolare chiusa, delimitata da due segmenti paralleli. Questo concetto risulterà più chiaro nella sezione dedicata all'estrazione e alla classificazione delle facce. Ci servono quindi altri due vertici per individuare il secondo segmento, parallelo al primo. Consideriamo lo scenario di Figura 2.9.

Sia $\vec{CD}$ il segmento noto, dove C e D sono le due giunzioni con i muri. Il nodo C è collegato, tramite archi WALL già esistenti, ai soli vicini A e F ; analogamente, il nodo D è collegato a B e H . Per individuare quale dei due vicini di C appartenga al segmento parallelo cercato, si calcola il prodotto scalare tra i versori (vettori normalizzati) di $\vec{CD}$ e di ciascun vicino: un valore prossimo

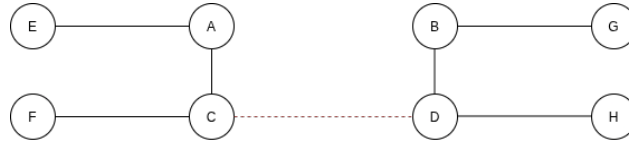


Figura 2.9: Singolo segmento

a 0 indica perpendicolarità tra i due segmenti. Applicando questo criterio ai vicini di C : il vettore $\vec{CF}$ risulta pressoché opposto a $\vec{CD}$ (prodotto scalare ≈ -1) e viene scartato; il vettore $\vec{CA}$ risulta invece perpendicolare a $\vec{CD}$ (prodotto scalare ≈ 0), individuando così il primo vertice candidato, A . Ripetendo lo stesso criterio sui vicini di D (B e H), si individua analogamente il secondo vertice candidato, B . Il secondo segmento cercato, parallelo a $\vec{CD}$, è quindi $\vec{AB}$ (Figura 2.10).

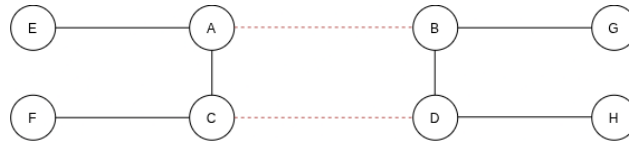


Figura 2.10: Doppio segmento

Per chiudere la faccia rettangolare della porta è quindi sufficiente aggiungere i due nuovi archi $\vec{CD}$ e $\vec{AB}$, entrambi con tag DOOR, senza dover introdurre alcun nuovo vertice.

Alla fine, ignorando per il momento le finestre, il procedimento di ricostruzione delle porte produce la rappresentazione mostrata in Figura 2.11: dopo la fase di snapping disponevamo di 88 vertici e 88 archi; dopo la ricostruzione delle porte otteniamo 10 nuovi archi (98 archi totali). Il dato è coerente, poiché sono presenti 5 porte, ciascuna delle quali richiede l'aggiunta di 2 nuovi archi.

2.4.2 Ricostruzione finestre

Resta da affrontare la ricostruzione delle finestre. Nel modello di caso di studio una finestra è rappresentata da decine di segmenti e punti; in altri modelli, tuttavia, le finestre possono presentarsi in forme completamente diverse. Non è possibile trattare esplicitamente ogni possibile scenario di rappresentazione, per cui è necessario un approccio più generale rispetto a quello utilizzato per le porte.

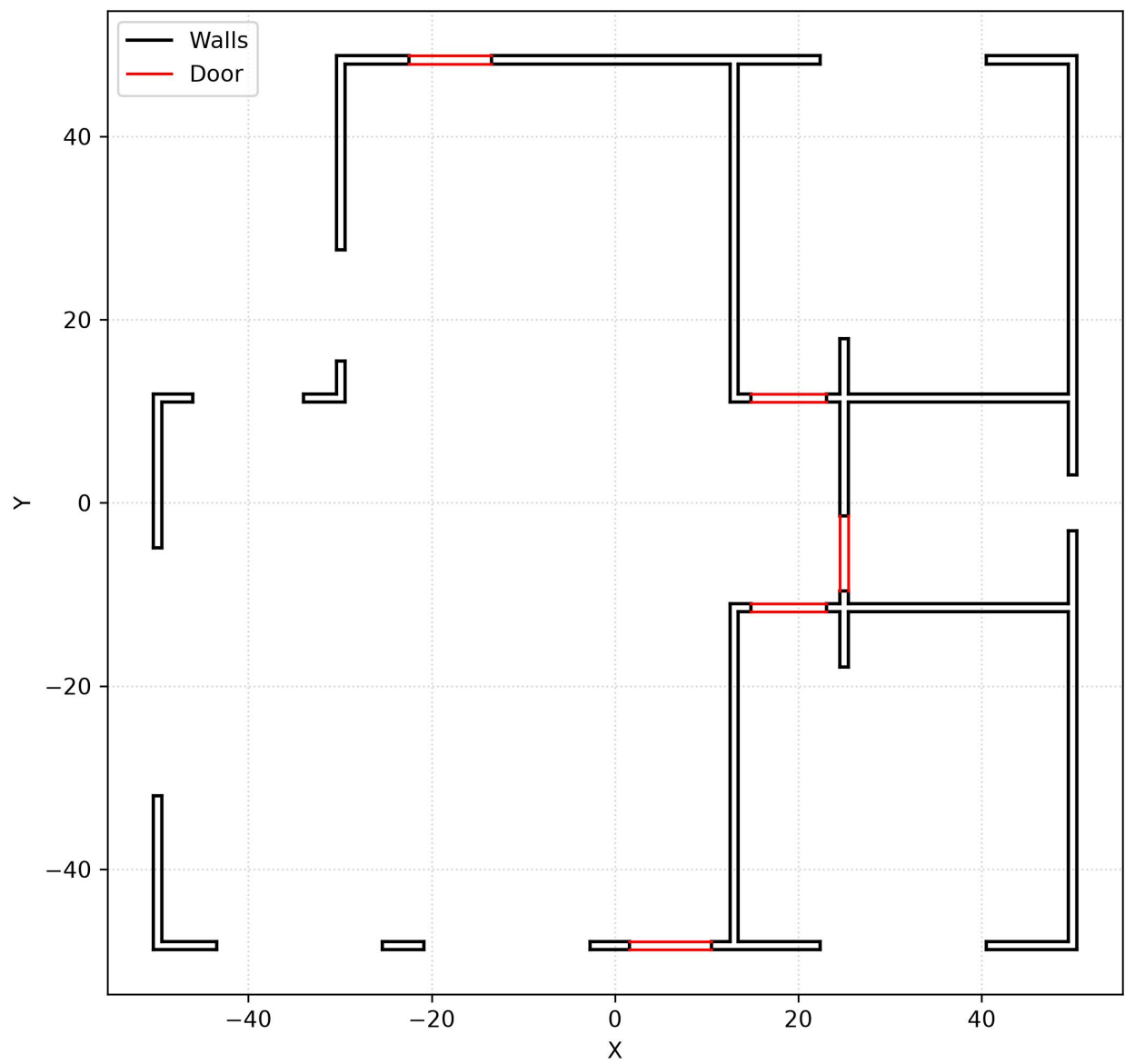


Figura 2.11: Le porte adesso sono rappresentate da due segmenti paralleli

L'approccio adottato si basa sul clustering spaziale e sull'estrazione del bounding box. I punti campionati lungo i segmenti di ciascuna finestra vengono raggruppati in cluster: ogni cluster conterrà un numero variabile di punti, che non è rilevante ai fini dell'algoritmo, e rappresenterà una finestra distinta. Poiché interessa conoscere unicamente il punto di inizio e di fine di ciascuna finestra, per ogni cluster viene calcolato il bounding box, da cui si estrae uno dei due lati lunghi. Il segmento così ottenuto, che congiunge le due giunzioni con i muri, viene trattato esattamente come il segmento di partenza usato per le porte: si applica quindi lo stesso identico procedimento descritto in precedenza per ricostruire il secondo segmento parallelo e chiudere la faccia. Al termine, anche ogni finestra risulta rappresentata da una faccia chiusa.

L'algoritmo utilizzato per il clustering è DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [4], implementato tramite la libreria SimpleDBSCAN. Sarà cruciale impostare correttamente due parametri: il numero di sample generati lungo ciascun segmento di finestra (campionati in modo lineare lungo il segmento), e la distanza massima ε entro cui due sample sono considerati appartenenti allo stesso cluster. È necessario valutare un compromesso tra questi due parametri: con un numero elevato di sample il costo computazionale cresce, mentre con un numero troppo basso aumenta il rumore nei cluster risultanti. Analogamente, se la tolleranza ε è troppo alta si rischia che due cluster distinti finiscano per sovrapporsi; se è troppo bassa, si rischia di ottenere un cluster per ciascun punto.

Dopo aver calcolato i bounding box e ricavato i relativi segmenti, è possibile ricostruire tutte le finestre. La rappresentazione completa di muri, porte e finestre risultante è mostrata in Figura 2.13.

Il risultato di questa fase è un grafo completo di muri, porte e finestre, pronto per essere utilizzato nella costruzione del *Planar Straight-Line Graph* e nell'estrazione delle facce.

2.5 Estrazione e classificazione delle facce

2.5.1 Grafo planare embedded e struttura dati Half-Edge

Dopo che abbiamo ricostruito le fessure tra i muri, la geometria è rappresentata dagli stessi vertici (univoci) ma con degli archi in più. L'insieme di tutti gli archi sono etichettati come WALL, DOOR o WINDOW. A partire da questa rappresentazione, l'obiettivo è costruire un grafo planare embedded

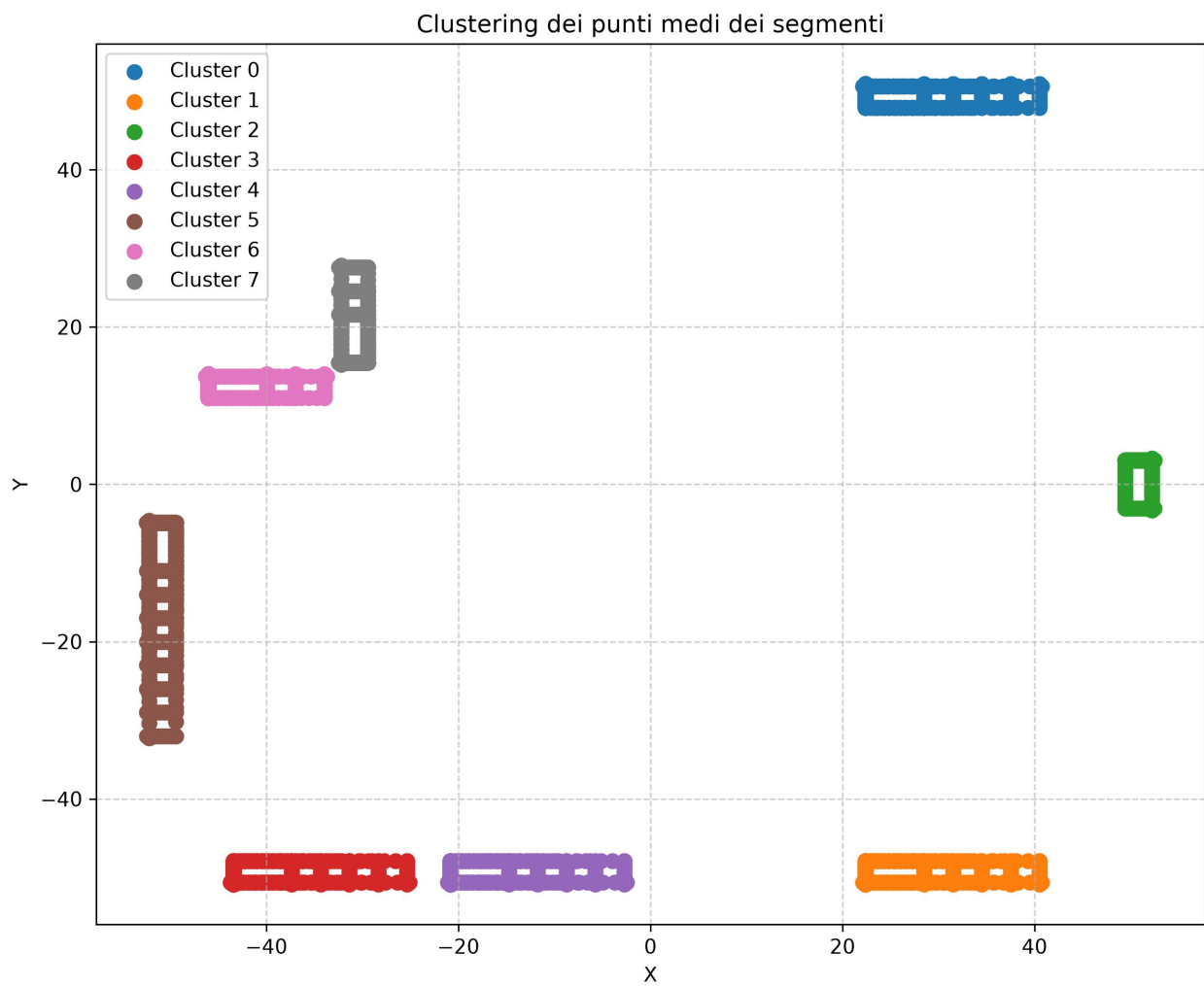


Figura 2.12: Nel nostro caso, con 15 sample per segmento e tolleranza $\varepsilon = 2$ otteniamo 8 cluster distinti corrispondenti alle 8 finestre presenti nel modello.

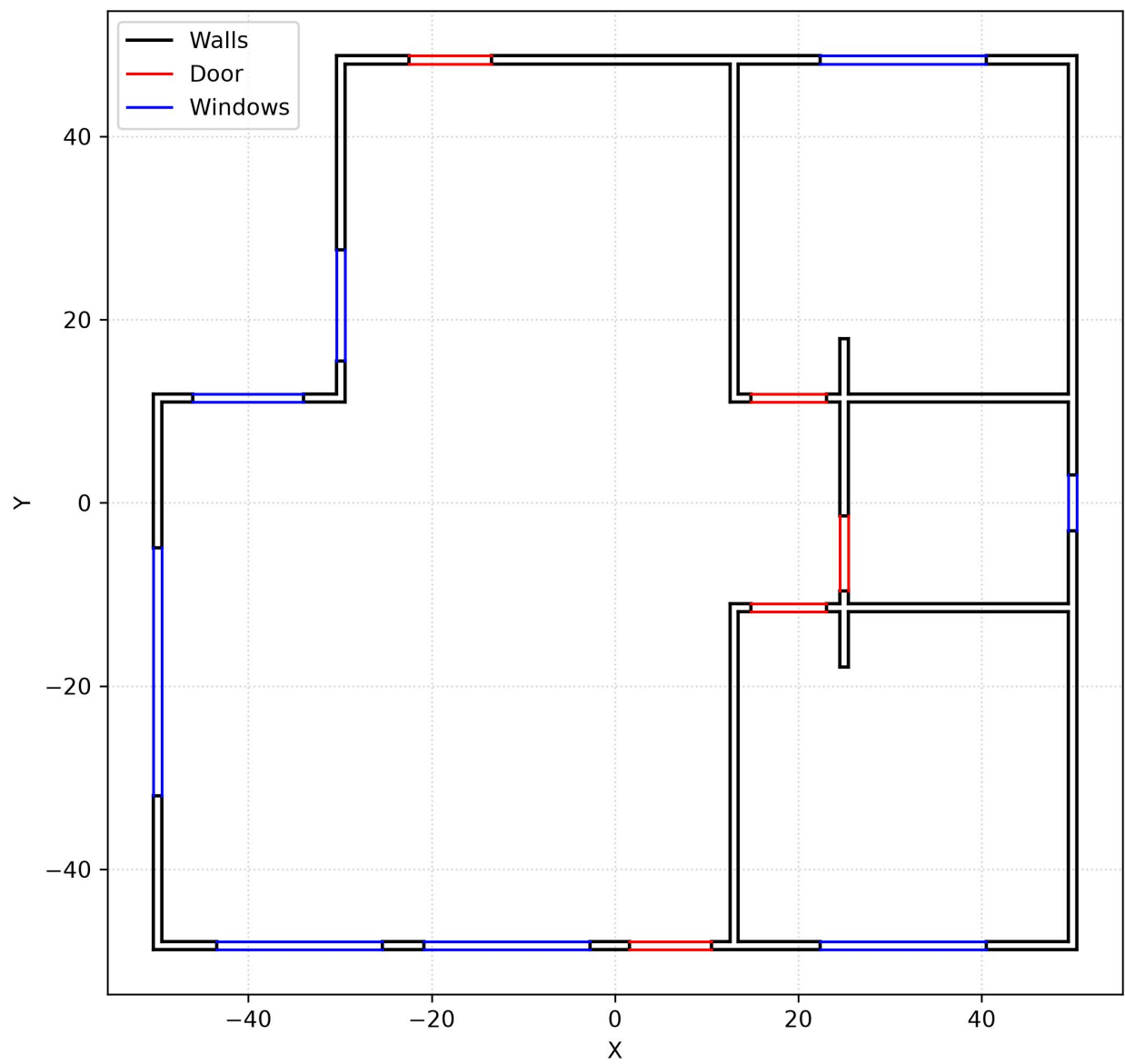


Figura 2.13: Ricostruzione completa dei varchi: muri, porte e finestre.

da cui estrarre le facce poligonali, che costituiranno la base per la successiva triangolazione ed estrusione.

Un grafo planare è un grafo che *può* essere disegnato su una superficie piana (ad esempio un foglio di carta) senza che i suoi archi si incrocino; non tutti i grafi sono planari. Un grafo planare embedded (o grafo piano) è invece un grafo planare che è stato *effettivamente* mappato su un piano, in modo tale che due archi non si intersechino se non nei loro vertici comuni.

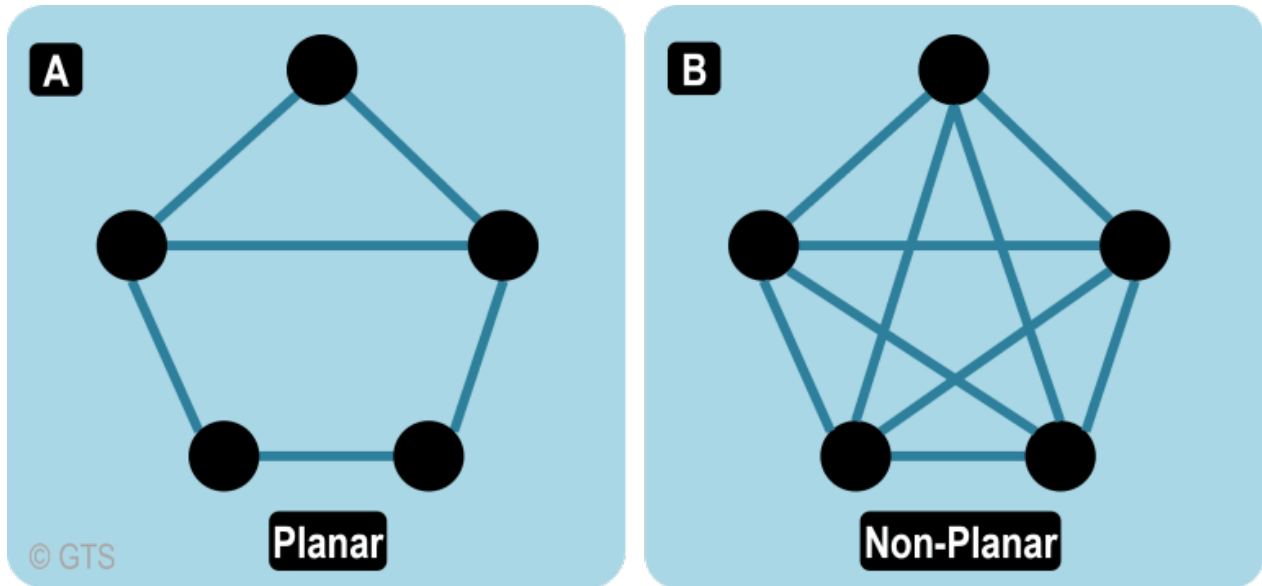


Figura 2.14: Grafo planare e non planare

Un grafo planare rettilineo (*Planar Straight-Line Graph*, PSLG) è un grafo planare embedded i cui archi sono rettilinei anziché curvi.

Esistono tre strutture dati ben note per rappresentare un PSLG: Winged-Edge, Half-Edge e Quad-Edge. In questo lavoro si è scelto di adottare la struttura Half-Edge [6, 5], messa a disposizione direttamente dalla libreria CGAL utilizzata per la costruzione del grafo.

La struttura dati Half-Edge, chiamata anche *doubly-linked edge list*, è una rappresentazione a grafo comunemente utilizzata in computer grafica e geometria computazionale per memorizzare mesh poligonali. Una mesh poligonale può essere vista come un grafo, costituito da vertici, archi e facce, ciascuna definita come un ciclo ordinato di vertici.

La struttura Half-Edge memorizza esplicitamente ogni arco come una coppia di archi orientati in direzioni opposte, detti *twin* (gemelli). Ogni arco mantiene un riferimento al proprio gemello,

oltre ai riferimenti all'arco precedente e successivo lungo la stessa faccia. Ogni vertice memorizza la propria posizione e un riferimento a uno degli archi uscenti da esso; ogni faccia memorizza a sua volta un riferimento a uno degli archi che la delimitano. La struttura risulta quindi composta da tre insiemi di record: vertici, facce e archi.

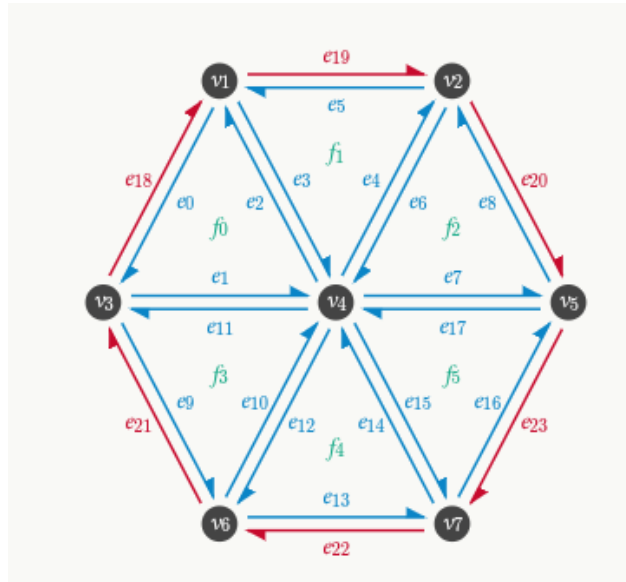


Figura 2.15: Visualizzazione Half-edge

Una volta costruito l'Half-Edge, il nostro output non è più un semplice elenco di segmenti, ma una raccolta di poligoni 2D (facce) distinti ed espliciti. A partire da questa rappresentazione è possibile procedere con l'estrazione e la classificazione delle facce, descritte nella sezione successiva.

2.5.2 Costruzione del PSLG con CGAL

Per implementare la struttura dati descritta, si è scelto di utilizzare la libreria CGAL [1], che fornisce la classe `CGAL::Arrangement_2`. Questa classe rappresenta un PSLG e fornisce accesso a vertici, half-edge e facce tramite appositi iteratori.

La costruzione dell'arrangement avviene passando a CGAL la lista dei vertici univoci (prodotti dalla fase di vertex snapping) e la lista degli archi etichettati. CGAL restituisce un oggetto `Arrangement_2` che contiene la lista delle facce.

```
CGAL::Arrangement_2 build_arrangement(
    const std::vector<glm::dvec2>& vertices,
    const std::vector<Edge>& edges
);

std::vector<Face> extract_faces(const Arrangement& arr);
```

Una faccia è rappresentata come:

```
class Face
{
    std::vector<glm::dvec2> vertices;
    std::vector<SegmentLayer> edge_layers;
    FaceType type;
};
```

È importante notare che l'arrangement include sempre una faccia illimitata (`unbounded_face()`), che rappresenta l'esterno del disegno e che deve essere esclusa dalla classificazione.

2.5.3 Classificazione delle facce

Una volta costruito l'arrangement, è possibile iterare sulle facce e classificarle in base agli archi.

La classificazione si basa sull'etichetta assegnata: una faccia è classificata come DOOR se e solo se ha esattamente 4 archi, di cui 2 etichettati come DOOR e 2 come WALL; una faccia è classificata come WINDOW se e solo se ha esattamente 4 archi, di cui 2 etichettati come WINDOW e 2 come WALL; una faccia è classificata come WALL se tutti i suoi archi sono etichettati come WALL; tutte le altre facce sono classificate come FLOOR (pavimento). Se il modello contiene più stanze, ciascuna di esse sarà rappresentata da una faccia FLOOR separata.

La classificazione è fondamentale perché l'estrusione viene effettuata in modo differenziato a seconda del tipo di faccia, come vedremo successivamente.

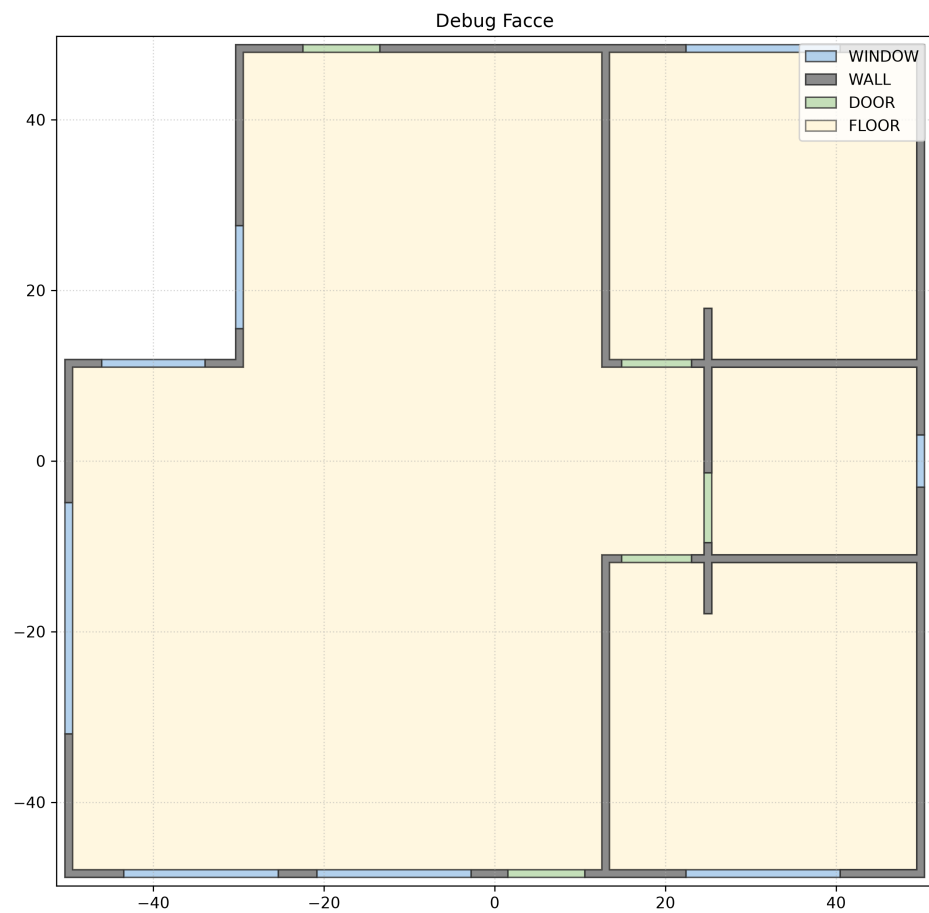


Figura 2.16: Visualizzazione facce

2.6 Triangolazione ed estrusione

Per ottenere i vertici e gli indici necessari per costruire la mesh, dobbiamo eseguire delle triangolazioni sulle facce. A tale scopo si utilizza l'algoritmo *Constrained Delaunay Triangulation*: presa in input una faccia sotto forma di poligono chiuso, restituisce un vettore di triangoli che la ricoprono. Ogni triangolo è rappresentato da 3 vertici, e per ogni vertice vengono calcolati posizione, normale e coordinate di texture.

```
struct Vertex
{
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 text_coord;
};
```

Il concetto di estrusione è molto semplice: dato un poligono (una faccia) 2D, lo si vuole trasformare in un poligono 3D. Il procedimento consiste nel generare gli stessi vertici a un'altezza diversa: per ogni lato della faccia, si considerano i due vertici ad altezza y_0 , si duplicano traslandoli alla nuova altezza y_1 , e si utilizzano i quattro vertici così ottenuti per creare un quad verticale.

Un quad ha sempre 4 vertici: *Bottom-Left* (BL), *Bottom-Right* (BR), *Top-Left* (TL) e *Top-Right* (TR), di conseguenza un quad viene suddiviso in 2 triangoli. Per ciascun vertice del quad si calcola inoltre il vettore normale, perpendicolare alla superficie del quad stesso: l'ordine con cui i vertici vengono collegati per formare i triangoli (*winding order*) determina il verso della normale. Percorrendo i vertici in senso antiorario (*counter-clockwise*, CCW) si ottiene una normale rivolta verso l'esterno, mentre percorrendoli in senso orario (*clockwise*, CW) la normale risulta rivolta verso l'interno; è quindi fondamentale prestare attenzione all'ordine dei vertici in fase di costruzione, per ottenere in ciascun caso il verso desiderato.

Le operazioni di triangolazione ed estrusione vengono applicate in modi diversi a seconda del tipo di faccia (pavimento, soffitto, muro, porta o finestra). È importante notare che, fino a questo punto, si è lavorato su poligoni 2D, con le facce iniziali poste ad altezza 0; da queste è ora necessario ricavare i corrispondenti poligoni 3D, ottenuti estrudendo le facce fino all'altezza del soffitto h .

2.6.1 Pavimento e soffitto

Una prima ipotesi naturale sarebbe rappresentare il pavimento tramite le facce classificate come FLOOR. Questo approccio, tuttavia, non è il più robusto: esistono modelli CAD dove non è presente la porta d'ingresso di una stanza o di un appartamento, lasciando l'ambiente aperto verso l'esterno. In questi casi la faccia FLOOR non esiste come entità separata, poiché viene inglobata nella cosiddetta *unbounded face* (la faccia esterna e illimitata generata automaticamente da CGAL per rappresentare tutto ciò che si trova al di fuori della geometria arrangiata).

L'approccio più robusto consiste quindi nel calcolare il bounding box dell'intera geometria e nel creare una singola faccia corrispondente a tale rettangolo. Il soffitto è ottenuto dalla stessa faccia, traslata all'altezza h e con normale invertita (rivolta verso il basso).

```
floor_face->triangulate(0.f);  
floor_face->triangulate(ceil_height);
```

2.6.2 Muri

Per i muri sarebbe in linea di principio possibile triangolare sia la faccia inferiore (ad altezza pavimento) sia la faccia superiore, traslata all'altezza del soffitto h . Questo, tuttavia, risulterebbe uno spreco: le facce di pavimento e soffitto, calcolate come descritto sopra, già chiudono la geometria superiormente e inferiormente. È quindi sufficiente estrarre le facce dei muri, senza ulteriori triangolazioni.

```
for(const auto& face : wall_faces)  
    face.extrude(0.f, ceil_height);
```

2.6.3 Porte

Una porta viene rappresentata come muro pieno a partire dall'altezza dello stipite s fino al soffitto, ad altezza h ; al di sotto dello stipite l'apertura resta libera. A differenza dei muri, in questo caso è necessario triangolare la faccia orizzontale posta all'altezza dello stipite s , poiché non esiste alcuna faccia di pavimento o soffitto che la copra a quella quota. La faccia ad altezza h , invece, non va

triangolata: quella porzione è già coperta dalla faccia del soffitto. È quindi necessario sia triangolare che estrarre.

```
for(const auto& face : door_faces)
{
    face.triangulate(door_height);
    face.extrude(door_height, ceil_height);
}
```

2.6.4 Finestre

La finestra è rappresentata come un'apertura al centro del muro: si distinguono quindi una porzione di muro inferiore, dal pavimento fino al davanzale (`window_sill`), e una porzione di muro superiore, dall'architrave (`window_height`) fino al soffitto. Entrambe le porzioni richiedono una triangolazione e un'estrusione: la prima triangolata al davanzale ed estrusa dal pavimento fino a quella quota, la seconda triangolata all'architrave ed estrusa fino al soffitto.

```
for(const auto& face : window_faces)
{
    face.triangulate(window_sill);
    face.extrude(0.0f, window_sill);

    face.triangulate(window_height);
    face.extrude(window_height, ceil_height);
}
```

2.7 Esportazione

Come ultimo passo, la mesh viene esportata come file GLTF. Vengono esportate le informazioni relative alla mesh: vertici (posizioni, normali, coordinate texture), indici e primitive. È necessario, in particolare, differenziare all'interno della mesh stessa quali primitive (triangoli) corrispondono al pavimento e quali invece appartengono a muri e soffitto; questa suddivisione viene realizzata

sfruttando gli indici. Durante la procedura di generazione della mesh vengono infatti mantenute separatamente le seguenti informazioni:

```
auto mesh_vertices      = std::vector<Vertex>{};
auto mesh_floor_indices = std::vector<u32>{};
auto mesh_wall_indices  = std::vector<u32>{};
```

I vertici sono sempre gli stessi, ma è possibile mappare un intervallo di indici per il pavimento e un intervallo distinto per muri e soffitto. Questo è necessario perché, una volta importato il modello in Blender, occorre poter distinguere il pavimento dal resto della geometria, per poter applicare materiali differenti.

2.7.1 Gestione degli asset tramite placeholder

Attualmente le porte e le finestre sono rappresentate come semplici buchi nei muri, ma questo non è sufficiente: si vuole poter inserire, in corrispondenza di quelle aperture, dei modelli 3D di porte e finestre.

Una prima ipotesi era inserire tali asset direttamente nel processo di generazione della mesh, ma questo approccio risulta poco conveniente. Se, ad esempio, in un appartamento sono presenti 10 porte identiche, la relativa geometria verrebbe duplicata 10 volte, aumentando notevolmente la dimensione del file GLTF, tanto più se si considera che un modello dettagliato può contare centinaia di migliaia di triangoli. Inoltre, per cambiare l'asset di una porta o modificarne l'orientamento, sarebbe necessario rieseguire l'intera pipeline; sarebbe infine necessario preservare anche i materiali originali del modello scaricato, complicando ulteriormente il processo.

La soluzione più flessibile e mantenibile è un approccio a placeholder, gestito interamente in Blender. La fase di classificazione delle facce viene quindi estesa: quando una faccia viene riconosciuta come porta o finestra, se ne calcola e serializza un *oriented bounding box* (OBB) dell'apertura, memorizzandone centro, larghezza, altezza, spessore, rotazione e tipo. In questo modo non vengono esportati placeholder geometrici, ma unicamente i dati delle aperture.

Una volta raccolte le informazioni sulle aperture di porte e finestre, queste vengono esportate in un file JSON separato, che verrà usato da Blender per posizionare i rispettivi modelli 3D all'interno della scena.

Di seguito è mostrato un esempio di struttura:

```
{  
  "openings": [  
    {  
      "type": "DOOR",  
      "center": [12.5, 0.0, 3.2],  
      "width": 0.9,  
      "height": 2.1,  
      "thickness": 0.1,  
      "rotation": 90.0  
    },  
    {  
      "type": "WINDOW",  
      "center": [8.3, 0.0, 1.5],  
      "width": 1.2,  
      "height": 1.4,  
      "thickness": 0.1,  
      "rotation": 0.0  
    }  
  ]  
}
```

3. Generazione della scena e rendering in Blender

3.1 Script Python di generazione scena Blender

3.2 Setup scena in Blender: materiali, luci, camera, Cycles

4. Risultati e valutazione

5. Conclusioni

Qui andrebbero le tue conclusioni.

Ringraziamenti

Qui i ringraziamenti, se lo desideri.

Bibliografia

- [1] *CGAL, Computational Geometry Algorithms Library*. URL: <https://www.cgal.org>.
- [2] Codelibs. *libdxfrw: A C++ library to read/write DXF files*. URL: <https://github.com/Codelibs/libdxfrw>.
- [3] *DWG vs DXF: differenze e confronto*. URL: <https://www.adobe.com/creativecloud/file-types/image/comparison/dwg-vs-dxf.html>.
- [4] Martin Ester et al. «A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise». In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*. AAAI Press, 1996.
- [5] UIUC CS418. *Half-Edge Data Structure*. URL: <https://cs418.cs.illinois.edu/website/text/halfedge.html>.
- [6] Jerry Y. *Half-Edge Data Structure*. URL: <https://jerryyin.info/geometry-processing-algorithms/half-edge/>.